

1. Perform Complete Data Wrangling on a Student Dataset

Code:

```
import pandas as pd

try:

    # Read CSV

    df = pd.read_csv("students.csv")

except FileNotFoundError:

    print("students.csv not found. Creating a sample DataFrame for demonstration.")

    data = {

        'StudID': [101, 102, 103, 104, 105, 101, 106],

        'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Alice', None],

        'Dept': ['CSE', 'EEE', 'CSE', None, 'MECH', 'CSE', 'EEE'],

        'Marks': [85, 90, 78, 92, None, 85, 65]

    }

    df = pd.DataFrame(data)

# Remove duplicate records

df = df.drop_duplicates()

# Handle missing values

df["Name"] = df["Name"].fillna("Unknown")

df["Marks"] = df["Marks"].fillna(df["Marks"].mean())

# Rename columns

df.rename(columns={

    "StudID": "StudentID",

    "Dept": "Department"

}, inplace=True)

# Change data types

df["StudentID"] = df["StudentID"].astype(int)

df["Marks"] = df["Marks"].astype(float)

# Save cleaned dataset

df.to_csv("students_cleaned.csv", index=False)

print(df)
```

Output:

	StudentID	Name	Department	Marks
0	101	Alice	CSE	85.0
1	102	Bob	EEE	90.0
2	103	Charlie	CSE	78.0
3	104	David	None	92.0
4	105	Eve	MECH	82.0
6	106	Unknown	EEE	65.0

2. Merge Student, Department, and Marks Datasets

Code:

```
import pandas as pd

import io

# Create data for students.csv

students_csv_data = """StudentID,Name,DepartmentID
101,Alice,1
102,Bob,2
103,Charlie,1
104,David,3
105,Eve,2
"""

students = pd.read_csv(io.StringIO(students_csv_data))

# Create data for departments.csv

departments_csv_data = """DepartmentID,DepartmentName
1,CSE
2,EEE
3,MECH
"""

departments = pd.read_csv(io.StringIO(departments_csv_data))

# Create data for marks.csv

marks_csv_data = """StudentID,Course,Mark
101,Math,85
101,Physics,70
```

102,Chemistry,90

103,Math,75

104,Biology,88

105,English,92

''''

```
marks = pd.read_csv(io.StringIO(marks_csv_data))
```

```
# Merge Student with Marks
```

```
data = pd.merge(students, marks, on="StudentID")
```

```
# Merge with Department
```

```
report = pd.merge(data, departments, on="DepartmentID")
```

```
print(report)
```

```
report.to_csv("student_report.csv", index=False)
```

Output:

	StudentID	Name	DepartmentID	Course	Mark	DepartmentName
0	101	Alice	1	Math	85	CSE
1	101	Alice	1	Physics	70	CSE
2	102	Bob	2	Chemistry	90	EEE
3	103	Charlie	1	Math	75	CSE
4	104	David	3	Biology	88	MECH
5	105	Eve	2	English	92	EEE

3. Convert Data Between Multiple Formats

Code:

```
import pandas as pd
```

```
import io
```

```
# Create CSV data
```

```
students_csv = '''StudentID,Name,Department,Marks
```

```
101,Alice,CSE,85
```

```
102,Bob,EEE,90
```

```
103,Charlie,CSE,75
```

```
104,David,MECH,88
```

```
105,Eve,EEE,92
```

```
''''
```

```

# Read CSV
df = pd.read_csv(io.StringIO(students_csv))

# Convert to JSON
df.to_json("students.json", orient="records", indent=4)

# Convert to XML
df.to_xml("students.xml", index=False)

# Read generated JSON
json_data = pd.read_json("students.json")

# Read generated XML
xml_data = pd.read_xml("students.xml")

print("JSON Data")

print(json_data)

print("\nXML Data")

print(xml_data)

```

Output:

JSON Data

	StudentID	Name	Department	Marks
0	101	Alice	CSE	85
1	102	Bob	EEE	90
2	103	Charlie	CSE	75
3	104	David	MECH	88
4	105	Eve	EEE	92

XML Data

	StudentID	Name	Department	Marks
0	101	Alice	CSE	85
1	102	Bob	EEE	90
2	103	Charlie	CSE	75
3	104	David	MECH	88
4	105	Eve	EEE	92

4. Analyze Student Performance Using Pandas

```
import pandas as pd

import io

# Create data

students_csv = """StudentID,Name,Marks

101,Alice,85

102,Bob,90

103,Charlie,75

104,David,88

105,Eve,92

"""

# Read CSV

df = pd.read_csv(io.StringIO(students_csv))

# Statistics

print("Highest Marks :", df["Marks"].max())

print("Lowest Marks :", df["Marks"].min())

print("Average Marks :", df["Marks"].mean())

print("Median Marks :", df["Marks"].median())

print("Standard Deviation :", df["Marks"].std())

# Grade Assignment

def grade(mark):

    if mark >= 90:

        return "A"

    elif mark >= 80:

        return "B"

    elif mark >= 70:

        return "C"

    elif mark >= 50:

        return "D"

    else:

        return "F"
```

```

df["Grade"] = df["Marks"].apply(grade)

# Top Performer

top_students = df[df["Marks"] == df["Marks"].max()]

print("\nStudent Report")

print(df)

print("\nTop Performer")

print(top_students)

```

Output:

```

Highest Marks : 92
Lowest Marks : 75
Average Marks : 86.0
Median Marks : 88.0
Standard Deviation : 6.670832032063167

```

Student Report

	StudentID	Name	Marks	Grade
0	101	Alice	85	B
1	102	Bob	90	A
2	103	Charlie	75	C
3	104	David	88	B
4	105	Eve	92	A

Top Performer

	StudentID	Name	Marks	Grade
4	105	Eve	92	A

5. Read Data from an SQLite Database and Generate Reports

Code:

```

import sqlite3

import pandas as pd

# Connect to SQLite database

conn = sqlite3.connect("college.db")

cursor = conn.cursor()

# Create table

cursor.execute("""

```

```

CREATE TABLE IF NOT EXISTS Students(
StudentID INTEGER,
Name TEXT,
Department TEXT,
Marks INTEGER)
)

# Insert records

cursor.execute("INSERT INTO Students VALUES(101,'Alice','CSE',85)")
cursor.execute("INSERT INTO Students VALUES(102,'Bob','EEE',90)")
cursor.execute("INSERT INTO Students VALUES(103,'Charlie','CSE',75)")
cursor.execute("INSERT INTO Students VALUES(104,'David','MECH',88)")
cursor.execute("INSERT INTO Students VALUES(105,'Eve','EEE',92)")

conn.commit()

# Read records

df = pd.read_sql("SELECT * FROM Students", conn)

print("All Students")

print(df)

# Filter records

high_marks = pd.read_sql(
    "SELECT * FROM Students WHERE Marks > 80",
    conn
)

print("\nStudents Scoring Above 80")

print(high_marks)

# Export to CSV

high_marks.to_csv("high_marks.csv", index=False)

conn.close()

```

Output:

All Students

	StudentID	Name	Department	Marks
0	101	Alice	CSE	85
1	102	Bob	EEE	90
2	103	Charlie	CSE	75
3	104	David	MECH	88
4	105	Eve	EEE	92

Students Scoring Above 80

	StudentID	Name	Department	Marks
0	101	Alice	CSE	85
1	102	Bob	EEE	90
2	104	David	MECH	88
3	105	Eve	EEE	92

6. Create and Analyze a Database Schema

Code:

```
import sqlite3

# Connect to SQLite
conn = sqlite3.connect("college.db")
cursor = conn.cursor()

# Create Student table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Student(
StudentID INTEGER PRIMARY KEY,
Name TEXT,
DepartmentID INTEGER)
""")

# Create Course table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Course(
CourseID INTEGER PRIMARY KEY,
CourseName TEXT)
```



```
""")
```

```
# Create Marks table
```

```
cursor.execute("""
```

```
CREATE TABLE IF NOT EXISTS Marks(
```

```
StudentID INTEGER,
```

```
CourseID INTEGER,
```

```
Marks INTEGER,
```

```
FOREIGN KEY(StudentID) REFERENCES Student(StudentID),
```

```
FOREIGN KEY(CourseID) REFERENCES Course(CourseID))
```

```
""")
```

```
# Insert Student Records
```

```
cursor.execute("INSERT INTO Student VALUES(101,'Alice',1)")
```

```
cursor.execute("INSERT INTO Student VALUES(102,'Bob',2)")
```

```
cursor.execute("INSERT INTO Student VALUES(103,'Charlie',1)")
```

```
# Insert Course Records
```

```
cursor.execute("INSERT INTO Course VALUES(1,'Python')")
```

```
cursor.execute("INSERT INTO Course VALUES(2,'Database')")
```

```
# Insert Marks
```

```
cursor.execute("INSERT INTO Marks VALUES(101,1,85)")
```

```
cursor.execute("INSERT INTO Marks VALUES(102,2,90)")
```

```
cursor.execute("INSERT INTO Marks VALUES(103,1,75)")
```

```
conn.commit()
```

```
# JOIN Query
```

```
query = """
```

```
SELECT Student.StudentID,
```

```
Student.Name,
```

```
Course.CourseName,
```

```
Marks.Marks
```

```
FROM Student
```

```
JOIN Marks
```

```
ON Student.StudentID = Marks.StudentID
```

JOIN Course

ON Course.CourseID = Marks.CourseID

"""

print("Student Report")

for row in cursor.execute(query):

print(row)

conn.close()

Output:

```
Student Report
(101, 'Alice', 'Python', 85)
(102, 'Bob', 'Database', 90)
(103, 'Charlie', 'Python', 75)
```

7. Data Validation and Error Detection

Code:

```
import pandas as pd
```

```
import io
```

```
# Create data
```

```
students_csv = """StudentID,Name,Email,Phone,Marks
```

```
101,Alice,alice@gmail.com,9876543210,85
```

```
102,Bob,bobgmail.com,987654321,90
```

```
103,Charlie,charlie@gmail.com,9876543212,-10
```

```
103,Charlie,charlie@gmail.com,9876543212,-10
```

```
104,,david@gmail.com,9876543213,88
```

```
"""
```

```
# Read CSV
```

```
df = pd.read_csv(io.StringIO(students_csv))
```

```
# Missing values
```

```
print("Missing Values")
```

```
print(df.isnull().sum())
```

```
# Duplicate records
```

```
print("\nDuplicate Records:", df.duplicated().sum())

# Invalid email
invalid_email = df[~df["Email"].str.contains("@", na=False)]
print("\nInvalid Email")
print(invalid_email)

# Invalid phone number
invalid_phone = df[df["Phone"].astype(str).str.len() != 10]
print("\nInvalid Phone Number")
print(invalid_phone)

# Negative marks
negative_marks = df[df["Marks"] < 0]
print("\nNegative Marks")
print(negative_marks)

# Clean dataset
df = df.drop_duplicates()
df = df[df["Marks"] >= 0]
df["Name"] = df["Name"].fillna("Unknown")
print("\nCleaned Dataset")
print(df)

df.to_csv("validated_students.csv", index=False)
```

Output:

```
Missing Values
StudentID    0
Name         1
Email        0
Phone        0
Marks        0
dtype: int64
```

Duplicate Records: 1

Invalid Email

	StudentID	Name	Email	Phone	Marks
1	102	Bob	bobgmail.com	987654321	90

Invalid Phone Number

	StudentID	Name	Email	Phone	Marks
1	102	Bob	bobgmail.com	987654321	90

Negative Marks

	StudentID	Name	Email	Phone	Marks
2	103	Charlie	charlie@gmail.com	9876543212	-10
3	103	Charlie	charlie@gmail.com	9876543212	-10

Cleaned Dataset

	StudentID	Name	Email	Phone	Marks
0	101	Alice	alice@gmail.com	9876543210	85
1	102	Bob	bobgmail.com	987654321	90
4	104	Unknown	david@gmail.com	9876543213	88

8. Build a Mini Data Wrangling Pipeline

Code:

```
import pandas as pd

import io

# CSV Data
csv_data = """StudentID,Name,Marks
101,Alice,85
102,Bob,90
"""

csv_df = pd.read_csv(io.StringIO(csv_data))

# Create JSON
csv_df.to_json("students.json", orient="records", indent=4)

# Create XML
csv_df.to_xml("students.xml", index=False)

# Read all formats
json_df = pd.read_json("students.json")
xml_df = pd.read_xml("students.xml")

# Combine datasets
combined = pd.concat([csv_df, json_df, xml_df])

# Remove duplicates
combined = combined.drop_duplicates()

# Handle missing values
combined = combined.fillna("Unknown")

print(combined)

# Save final output
combined.to_csv("final_dataset.csv", index=False)
```

Output:

	StudentID	Name	Marks
0	101	Alice	85
1	102	Bob	90

9. Employee Payroll Data Analysis

Code:

```
import pandas as pd

import io

# Employee data
employees_csv = """EmpID,Name,Department,BasicSalary,Allowance,Deduction
101,Alice,HR,30000,5000,2000
102,Bob,IT,40000,6000,2500
103,Charlie,HR,35000,4500,1500
104,David,Sales,45000,7000,3000
"""

# Read CSV
df = pd.read_csv(io.StringIO(employees_csv))

# Calculate Net Salary
df["NetSalary"] = df["BasicSalary"] + df["Allowance"] - df["Deduction"]

print(df)

# Highest Salary
highest = df[df["NetSalary"] == df["NetSalary"].max()]

print("\nHighest Salary Employee")

print(highest)

# Department-wise Salary Summary
summary = df.groupby("Department")["NetSalary"].sum()

print("\nDepartment-wise Salary Summary")

print(summary)

# Export Report
summary.to_csv("department_salary.csv")
```

Output:

	EmpID	Name	Department	BasicSalary	Allowance	Deduction	NetSalary
0	101	Alice	HR	30000	5000	2000	33000
1	102	Bob	IT	40000	6000	2500	43500
2	103	Charlie	HR	35000	4500	1500	38000
3	104	David	Sales	45000	7000	3000	49000

Highest Salary Employee

	EmpID	Name	Department	BasicSalary	Allowance	Deduction	NetSalary
3	104	David	Sales	45000	7000	3000	49000

Department-wise Salary Summary

Department

HR 71000

IT 43500

Sales 49000

Name: NetSalary, dtype: int64

10. Sales Data Analysis Dashboard

Code:

```
import pandas as pd
```

```
import io
```

```
# January Sales
```

```
jan = """Month,Product,Quantity,Price
```

```
January,Laptop,5,50000
```

```
January,Mouse,10,500
```

```
"""
```

```
# February Sales
```

```
feb = """Month,Product,Quantity,Price
```

```
February,Laptop,3,50000
```

```
February,Keyboard,8,1000
```

```
"""
```

```
# Read CSV data
```

```
jan_df = pd.read_csv(io.StringIO(jan))
```

```
feb_df = pd.read_csv(io.StringIO(feb))
```

```
# Merge datasets
```

```
sales = pd.concat([jan_df, feb_df])
```

```
# Calculate Revenue
```

```

sales["Revenue"] = sales["Quantity"] * sales["Price"]

print(sales)

# Total Revenue

print("\nTotal Revenue:", sales["Revenue"].sum())

# Monthly Revenue

monthly = sales.groupby("Month")["Revenue"].sum()

print("\nMonthly Revenue")

print(monthly)

# Top-selling Products

top_products = sales.groupby("Product")["Quantity"].sum().sort_values(ascending=False)

print("\nTop-selling Products")

print(top_products)

# Export Reports

monthly.to_csv("monthly_revenue.csv")

top_products.to_csv("top_products.csv")

```

Output:

	Month	Product	Quantity	Price	Revenue
0	January	Laptop	5	50000	250000
1	January	Mouse	10	500	5000
0	February	Laptop	3	50000	150000
1	February	Keyboard	8	1000	8000

Total Revenue: 413000

Monthly Revenue

Month

February 158000

January 255000

Name: Revenue, dtype: int64

Top-selling Products

Product

Mouse 10

Keyboard 8

Laptop 8

Name: Quantity, dtype: int64